

WEEK 5 PRACTICAL

QUESTION 1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Aim

To implement communication between a parent and child process using an anonymous pipe and measure the communication time.

Program

```
GNU nano 7.2                                practical5_task1.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <sys/time.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from Producer Process!";
    char buffer[100];

    struct timeval start, end;

    // Create anonymous pipe
    if (pipe(fd) == -1)
    {
        printf("Pipe creation failed!\n");
        return 1;
    }

    // Start time
    gettimeofday(&start, NULL);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    // Child Process (Consumer)
    if (pid == 0)
    {
        close(fd[1]); // Close write end

        read(fd[0], buffer, sizeof(buffer));

        printf("Consumer (Child) received: %s\n", buffer);

        close(fd[0]); // Close read end
    }

    // Parent Process (Producer)
    else
    {
        close(fd[0]); // Close read end

        printf("Producer (Parent) sent: %s\n", message);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]); // Close write end

        wait(NULL);

        // End time
        gettimeofday(&end, NULL);

        long time_taken =
            (end.tv_sec - start.tv_sec) * 1000000L +
            (end.tv_usec - start.tv_usec);

        printf("Communication Time: %ld microseconds\n", time_taken);
    }

    return 0;
}
```

Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was

Output

```
navaneeth@NAVANEETH:~$ mkdir practical_week5
navaneeth@NAVANEETH:~$ cd practical_week5
navaneeth@NAVANEETH:~/practical_week5$ nano practical5_task1.c
navaneeth@NAVANEETH:~/practical_week5$ gcc practical5_task1.c -o practical5_task1
navaneeth@NAVANEETH:~/practical_week5$ ls
practical5_task1  practical5_task1.c
navaneeth@NAVANEETH:~/practical_week5$ ./practical5_task1
Producer (Parent) sent: Hello from Producer Process!
Consumer (Child) received: Hello from Producer Process!
Communication Time: 329 microseconds
```

Observation

The parent process generated data and sent it through the pipe. The child process successfully received the data, and the communication time was measured.

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe.

QUESTION 2

Develop a program that executes the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Aim: To implement a pipeline between two commands using `fork()`, `pipe()`, `dup2()` and `exec()`.

Program

```
GNU nano 7.2 practical5_task2.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1)
    {
        printf("Pipe creation failed!\n");
        return 1;
    }

    // First Child - ls -l
    pid1 = fork();

    if (pid1 == 0)
    {
        close(fd[0]); // Close read end
        dup2(fd[1], STDOUT_FILENO); // Redirect output to pipe
        close(fd[1]);

        char *cmd1[] = {"ls", "-l", NULL};
        execvp(cmd1[0], cmd1);

        printf("Execution failed.\n");
        exit(1);
    }

    // Second Child - grep ".c"
    pid2 = fork();

    if (pid2 == 0)
    {
        close(fd[1]); // Close write end
        dup2(fd[0], STDIN_FILENO); // Read input from pipe
        close(fd[0]);

        char *cmd2[] = {"grep", ".c", NULL};
        execvp(cmd2[0], cmd2);

        printf("Execution failed.\n");
        exit(1);
    }

    // Parent Process
    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    printf("\nPipeline executed successfully.\n");
    return 0;
}
```

Output

```
navaneeth@NAVANEETH:~/practical_week5$ nano practical5_task1.c
navaneeth@NAVANEETH:~/practical_week5$ nano practical5_task2.c
navaneeth@NAVANEETH:~/practical_week5$ gcc practical5_task2.c -o practical5_task2
navaneeth@NAVANEETH:~/practical_week5$ ./practical5_task2
-rwxr-xr-x 1 navaneeth navaneeth 16408 Sep 18 05:00 practical5_task1
-rw-r--r-- 1 navaneeth navaneeth 1339 Sep 18 05:05 practical5_task1.c
-rwxr-xr-x 1 navaneeth navaneeth 16312 Sep 18 05:09 practical5_task2
-rw-r--r-- 1 navaneeth navaneeth 1106 Sep 18 05:09 practical5_task2.c

Pipeline executed successfully.
navaneeth@NAVANEETH:~/practical_week5$
```

Observation

The output of `ls -l` was passed through the pipe to `grep ".c"`, which displayed the matching C files.

Result

The shell command `ls -l | grep ".c"` was successfully implemented using system c